

HermesQA — Báo cáo tuần 30/09 → 07/10/2026

Sinh viên: Phí Thành Dương · Kỳ báo cáo: 30/09 – 07/10/2026 · Mã nguồn: github.com/zuberg0811/HermesQA (nhánh `main`)

1. Tuần này nói ngắn gọn

Tuần trước tôi dừng lại ở một con số không thoải mái: cấu hình "kết hợp công cụ tĩnh với AI" — vốn là ý tưởng chính của đồ án — chỉ đạt F1 53 %, thua xa cấu hình chỉ dùng AI. Số đó lại còn là số chấm lại offline, chưa phải chạy thật. Tuần này tôi đặt mục tiêu làm ba việc: chạy thật để biết sự thật, đi học xem người ta làm thế nào, và thử hệ thống trên pull request thật thay vì trên bộ lỗi tự tìm.

Kết quả tóm tắt: F1 của cấu hình kết hợp lên **87,2 %** (chạy thật), cấu hình chỉ AI lên **89,5 %**, số báo động giảm bốn đến năm lần. Nhưng phát hiện đáng giá nhất lại là một con số xấu: trên 15 pull request thật, hệ thống chỉ bắt được **1–2 trong 68 vấn đề** mà kỹ sư đã xác nhận. Nói cách khác, bộ đề tự tạo của tôi dễ hơn đời thật rất nhiều, và báo cáo cuối cùng sẽ phải nói thẳng điều này.

Khối lượng: 9 commit, 43 file thay đổi (+3 118 / –216 dòng), 35 kiểm thử tự động đều đạt. Mọi tài liệu (báo cáo tiến độ 21 trang, đề xuất cải tiến 9 trang, trang tổng quan) đã cập nhật và đẩy lên GitHub.

2. Đã làm gì, theo thứ tự thời gian

Chạy thật và tìm ra một lỗi "im lặng" (05/10)

Việc đầu tiên là chạy lại thật cấu hình kết hợp trên 35 PR. Kết quả thật ra tệ hơn số chấm lại: có 11 cảnh báo giả giống hệt nhau, đều là bandit B101 ("dùng `assert` trong code") trong file test — thứ mà tôi *tưởng* đã lọc từ tuần trước. Lần theo thì thấy hàm lọc có test riêng, test chạy xanh, nhưng chưa bao giờ được gọi trong đường chạy thật: lệnh sửa file hôm trước không khớp vì file dùng CRLF, và script không báo lỗi. Tôi gọi đây là bug D3 và rút ra một nguyên tắc ghi vào báo cáo: *test từng hàm lẻ không chứng minh hàm đó được gọi; test phải đi qua đúng đường mà sản phẩm chạy*.

Cùng ngày tôi sửa lại quy tắc gộp nhận xét giữa công cụ tĩnh và AI. Hai cách thử đầu đều làm mất 2–3 lỗi thật (gộp theo vị trí bắt kể nguồn thì gộp nhầm hai vai AI; để nhận xét nặng hơn làm chính thì AI tự chấm "thiếu test" là lỗi nặng và đề lên cảnh báo tĩnh). Cách giữ lại: công cụ tĩnh hấp thụ nhận xét AI cùng chỗ *nếu* AI không chấm nặng hơn; hai vai AI chỉ gộp khi cùng loại lỗi. Số chốt cho pipeline cũ sau bước này: chỉ AI 70,1 %, kết hợp 66,0 %.

Đi học Alibaba (05/10)

Tôi đọc kỹ Open Code Review — công cụ review PR bằng AI mà Alibaba dùng nội bộ hai năm rồi mở mã nguồn đầu năm nay — cùng bài báo và bộ dữ liệu AACR-Bench đi kèm. Chi tiết ở mục 3. Kết quả là một tài liệu đề xuất 7 cải tiến (có bản dành cho người không chuyên) và, vì thầy đã đồng ý, tôi triển khai cả 7 ngay trong ngày (mục 4).

Đo pipeline mới, và bị số liệu dạy cho vài bài (06/10)

Hạn mức miễn phí của Gemini (500 lượt/ngày cho mỗi model) cạn giữa chừng nên việc đo phải kéo sang hôm sau. Trong lúc chờ, tôi đọc dữ liệu trung gian của những case đã chạy và thấy ba chuyện:

- **Bước kiểm tra sự thật loại nhầm nhận xét đúng.** AI giám khảo bác bỏ "dùng `== None` thay vì `is None`" với lý do "code đã xử lý rồi", và bằng chứng nó dẫn ra là... chính dòng `if user == None:`. Lý luận vòng. Tôi thêm một ràng buộc cơ học: bằng chứng "đã xử lý ở chỗ khác" phải là một dòng *khác* dòng bị báo. Sau sửa, bước này không loại nhầm lẫn nào nữa.
- **Pilot trên PR thật chạy sai dữ liệu.** Bộ AACR-Bench đặt tên trường ngược với trực giác: `pr_source_commit` hóa ra là nhánh đích, không phải nhánh PR. Lần chạy đầu vì thế review diff đảo chiều, có PR 22 dòng mà diff lên tới 153 000 dòng. Tôi phát hiện khi so số dòng với dataset và hỏi lại GitHub API; bỏ toàn bộ lần chạy đó, sửa cách lấy diff, kiểm tra 68/68 vấn đề chuẩn đều nằm trong diff mới rồi mới chạy lại.
- **Công cụ tĩnh trên repo thật nói về những dòng không ai sửa.** 240 trong 275 cảnh báo tĩnh nằm ngoài phần thay đổi của PR — vừa gây nhiễu, vừa không đăng được vì GitHub không cho comment ngoài hunk. Thêm bộ lọc "chỉ trong phạm vi thay đổi" vào cả CLI lẫn worker.

Chốt số và viết lại tài liệu (07/10)

Chạy lại đủ bốn cấu hình bộ Python và hai cấu hình bộ đa ngôn ngữ với đúng model cũ để so sánh công bằng. Khi rà 15 cảnh báo giả còn lại của cấu hình kết hợp, 7 cái là hai công cụ (hadolint và semgrep) cùng báo một lỗi ở cùng một dòng nhưng gắn hai nhãn khác nhau nên không được gộp. Tôi đổi quy tắc: mọi cảnh báo tĩnh trên cùng một dòng là một comment, các luật khác dính kèm bên dưới. F1 kết hợp tăng 81,4 → 87,2 %; cái giá là mất một lỗi thật vì nhãn của cụm lấy theo cảnh báo chính — tôi ghi rõ đây là đánh đổi, không phải cải thiện thuần.

Cuối ngày: chạy giám khảo AI phân loại báo động giả, cập nhật báo cáo tiến độ, tài liệu đề xuất và trang tổng quan, thêm `tools/md2pdf.py` để mọi file PDF của đồ án cùng một định dạng.

3. Nghiên cứu giải pháp của Alibaba: Open Code Review

Nó là gì. Open Code Review (OCR) là công cụ review pull request bằng AI, viết bằng Go, giấy phép Apache-2.0, hơn 43 000 sao trên GitHub. Alibaba dùng nội bộ hai năm trước khi mở mã. Đi kèm là bài báo và AACR-Bench — 200 PR thật từ các dự án mã nguồn mở, 10 ngôn ngữ, 1 505 vấn đề do 80 kỹ sư xác nhận bằng tay. Tôi đã đọc phần chọn file, định vị nhận xét, toàn bộ lời nhắc, luật theo ngôn ngữ, bộ lọc kết quả, và bài báo.

Họ làm tốt điều gì

Điều tôi ấn tượng nhất không phải lời nhắc hay model, mà là họ **không tin AI ở những chỗ không cần tin**:

- *Định vị nhận xét bằng đoạn code chứ không bằng số dòng.* AI phải chép lại nguyên văn dòng code nó đang nói đến; phần mềm tự tìm dòng đó trong diff. Số dòng do AI đếm sai liên tục, đoạn code thì gần như không.
- *Bộ lọc kiểm tra sự thật mặc định "giữ".* Thay vì bắt AI viết lại toàn bộ nhận xét cho "chuẩn hơn" (cách tôi làm trước đây và đo được là có hại), họ chỉ cho phép loại bỏ khi có một trong hai căn cứ cụ thể, và không bao giờ loại lỗi bảo mật nặng.
- *Luật theo ngôn ngữ kèm mục "Do not report".* Danh sách những thứ đúng nhưng vô ích mà AI rất thích nói ("nên thêm docstring", "nên có type hint"). Đây là nguồn báo động giả lớn nhất của tôi.
- *Chọn file tắt định.* Sáu cổng lọc, danh sách loại trừ mặc định, đường dẫn bí mật không bao giờ gửi cho AI; file liên quan gom vào cùng một lượt để AI có đủ ngữ cảnh.
- *Đánh giá trên PR thật* bằng giám khảo AI bỏ phiếu — và họ công bố cả số xấu: agent tốt nhất của họ (Claude 4.5 Sonnet) chỉ đạt precision 39,9 %, recall 10,1 %.

Họ yếu ở đâu, hoặc khác tôi ở đâu

- **Không có công cụ tĩnh.** Toàn bộ dựa vào AI, kể cả những lỗi mà một luật tất định bắt được rẻ hơn, nhanh hơn và không bao giờ bị (SQL injection ghép chuỗi, khóa API lộ, `shell=True`). Trong bộ đo của tôi, công cụ tĩnh bắt được 12/13 lỗi bảo mật — ngang AI — với chi phí token bằng không.
- **Không có lớp kiểm chứng cơ học.** Bộ lọc kiểm tra sự thật của họ cũng là AI; AI tự kiểm AI. Tôi gặp đúng rủi ro đó (lý luận vòng ở trên) và phải thêm phần mềm đối chiếu bằng chứng.
- **Không có sandbox.** Agent của họ có công cụ đọc file tự do — hợp lý trong nội bộ Alibaba, nhưng tôi không muốn chạy mã lạ từ một PR công khai theo cách đó. HermesQA giữ container Docker không mạng, chỉ đọc.
- **Một vai duy nhất.** Đơn giản, nhưng bỏ sót góc nhìn kiểm thử và DevOps; ba vai SE/QA/DevOps của tôi bắt được nhóm lỗi test 4/4 mà công cụ tĩnh bắt 0/4.
- **Recall thấp ngay cả với model mạnh.** Đây không hẳn là điểm yếu riêng của họ mà là thực tế của bài toán: review PR thật khó hơn mọi benchmark tự tạo. Số pilot của tôi còn thấp hơn (recall 1,5–3 %) vì dùng model nhỏ.
- **Benchmark tự xây, tự chấm bằng AI** — có thiên vị "đồng ý với chính mình". Tôi gặp lại điều này khi dùng giám khảo nhỏ (mục 5).

Nhận định chung: OCR giải rất tốt phần *định vị, lọc và vận hành*; tôi học phần đó. Phần *lai công cụ tĩnh + sandbox + nhiều vai* là thứ họ không có và tôi giữ làm điểm khác biệt của đề án. Điều cả hai bên cùng chứng minh: trên PR thật, AI review còn xa mức thay được người.

4. Đề xuất cải tiến hệ thống học từ Alibaba — và kết quả sau khi làm

Bảy đề xuất đã nêu với thầy, nay đều đã triển khai (commit `d2c5a0b`, chỉnh thêm ở `2133adf`, `8e87c2e`). Tôi ghi lại từng cái với kết quả đo được, kể cả cái chưa thấy lợi.

#	Đề xuất	Đã làm thế nào	Kết quả đo được
1	Ghim nhận xét bằng đoạn code	AI phải trả <code>existing_code</code> ; phần mềm tìm lại dòng (khớp nguyên văn → bỏ dấu +/- → dòng dài nhất, ưu tiên dòng mới thêm)	39/39 nhận xét ghim được; F1 ở độ lệch 0 dòng bằng đúng độ lệch 3 dòng (trước mất 1)
2	Kiểm tra sự thật thay self-verify	AI chỉ được loại với căn cứ A (code không tồn tại) hoặc B (đã xử lý ở dòng khác); phần mềm kiểm bằng chứng; lỗi nặng được bảo vệ	Không mất lỗi thật nào (self-verify cũ mất 1 và thêm 5 báo giả). Nhưng cũng chưa loại được báo giả nào và tốn thêm 40 % token — <i>an toàn, chưa chứng minh được lợi ích với model nhỏ</i>
3	Danh sách kiểm tra theo loại file, có mục "Không báo"	9 file luật cho Python, JS/TS, Go, Java, Dockerfile, deps, CI, test	Đóng góp lớn nhất: báo động giả 27 → 6 (chỉ AI), 61 → 8 (kết hợp). Rủi ro: luật viết sau khi đã thấy bộ đề
4	Chia lô thay vì cắt bớt im lặng	Chia file theo ngân sách ký tự; file bị cắt liệt kê trong phần "Phạm vi review"	Chưa có case nào trong bộ đo chạm ngưỡng; có tác dụng trên PR thật (file 141 000 ký tự được báo rõ)
5	Loại trừ mặc định, chặn file bí mật	<code>.env</code> , khóa, lockfile, file sinh tự động không gửi cho AI; dòng gitleaks đánh dấu được che	Hoạt động đúng trên PR thật (3 lockfile bị bỏ qua, có ghi lý do)
6	Dấu vân tay nhận xét	Bấm file + loại + nội dung code; bảng <code>posted_comments</code> ; không đăng lại khi đẩy commit mới	Có test; chưa đo được trên GitHub thật vì chưa có credential
7	Nâng cấp cách đánh giá	Chấm ở độ lệch 0 dòng; giám khảo AI phân loại báo động giả; pilot trên PR thật	Chính ba công cụ này tìm ra các vấn đề ở mục 2

Ngoài bảy đề xuất, tuần này phát sinh thêm hai cải tiến từ số đo: lọc cảnh báo tĩnh theo phạm vi thay đổi, và gộp cảnh báo tĩnh cùng dòng (đều ở mục 2).

Những gì tôi cố tình không học theo: bỏ công cụ tĩnh (lý do ở mục 3), cho AI công cụ đọc file tự do không sandbox, và gộp ba vai thành một. Tôi cũng không bê nguyên lời nhắc dài của họ; chỉ lấy cấu trúc "Báo / Không báo".

5. Số liệu cuối kỳ

Bộ Python 35 PR, 36 lỗi tiềm; model `gemini-3.1-flash-lite` cho cả trước và sau để so sánh công bằng. "Đúng dòng" là chấm ở độ lệch 0.

Cấu hình	TP	FP	FN	Precision	Recall	F1 trước	F1 sau	Đúng dòng	Giây · token / PR
Chỉ công cụ tĩnh	21	9	15	70,0 %	58,3 %	62,2 %	63,6 %	54,5 %	101 · 0
Chỉ AI, kiểm tra sự thật	34	6	2	85,0 %	94,4 %	70,1 %	89,5 %	89,5 %	18,5 · 6 355
Chỉ AI, không kiểm chứng	34	6	2	85,0 %	94,4 %	69,4 %	89,5 %	89,5 %	12,2 · 4 532
Chỉ AI, self-verify cũ	33	11	3	75,0 %	91,7 %	70,1 %	82,5 %	82,5 %	18,6 · 8 656
Kết hợp (mặc định sản phẩm)	34	8	2	81,0 %	94,4 %	66,0 %	87,2 %	76,9 %	118 · 6 551

Bộ đa ngôn ngữ 18 PR: chỉ AI 85,7 % (trước 73,5), kết hợp 81,8 % (trước 72,0), recall 100 % cả hai.

PR thật — AACR-Bench, 15 PR Python dưới 210 dòng, 68 vấn đề do kỹ sư xác nhận, model `gemini-3.5-flash-lite` , giám khảo 3 phiếu:

Cấu hình	Nhận xét đưa ra	Trúng	Precision	Recall	F1
Chỉ AI	8	1	12,5 %	1,5 %	2,6 %
Kết hợp như cũ	286	2	0,7 %	2,9 %	1,1 %
Kết hợp, chỉ giữ cảnh báo trong phạm vi thay đổi	43	1	2,3 %	1,5 %	1,8 %

Giám khảo AI phân loại báo động giả (model khác model sinh, 3 phiếu): trên bộ Python, cả 20 báo động giả đều là nhận xét đúng nằm ngoài đáp án (thiếu test cho hàm mới, thiếu xử lý lỗi...); bộ đa ngôn ngữ 5/6 đúng và 1 bịa, tức 4 % tổng nhận xét. Tôi ghi kèm cảnh báo: giám khảo nhỏ có xu hướng gật — với model lớn hơn (chỉ đủ hạn mức thử một mẫu), 2/3 phiếu lại cho rằng nhận xét "thiếu test cho hàm ping" là không có căn cứ.

Ba điều tôi rút ra từ bảng số: báo động giả giảm nhiều là thật nhưng có mùi "học thuộc đề"; self-verify kiểu cũ nên bỏ hẳn; và khoảng cách giữa 89,5 % trên bộ đề với 2,6 % trên PR thật là thứ cần được nói thẳng trong báo cáo cuối, không nên giấu.

6. Kế hoạch thử nghiệm Firebase cho website demo

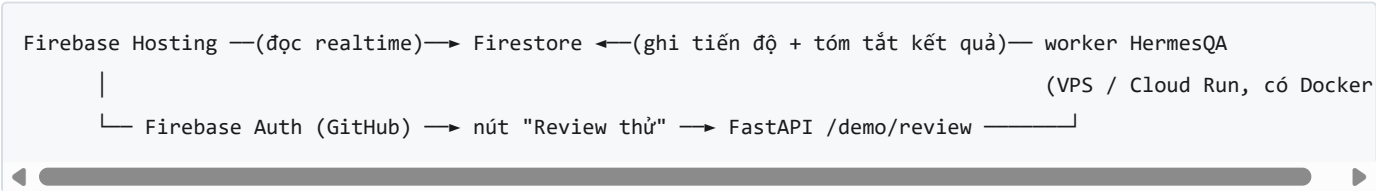
Sau khi hoàn thiện, tôi muốn có một website để trình bày đề án: người xem đăng nhập bằng GitHub, thấy danh sách PR đã review, bấm vào xem từng nhận xét, và có thể bấm "review thử" trên một repo mẫu rồi nhìn hệ thống chạy từng bước. Tôi đã xem Firebase giải quyết được gì cho việc này.

Firestore hợp với phần "mặt tiền":

- *Hosting* cho trang web (HTTPS, domain miễn phí, deploy một lệnh).
- *Authentication* với nhà cung cấp GitHub — có sẵn nút đăng nhập, không phải tự viết OAuth; token nhận được còn dùng được để liệt kê repo của người xem.
- *Firestore* với realtime listener: worker ghi tiến độ ("đang quét static → đang hỏi vai SE → đã đăng 5 nhận xét") và trang web tự cập nhật — đây là hiệu ứng demo tốt nhất mà Postgres cộng polling không cho được dễ.
- Gói miễn phí đủ cho quy mô demo.

Firestore không thay được phần lõi: worker và sandbox Docker không chạy được trên Cloud Functions, nên phần review vẫn ở một máy chủ riêng (VPS rẻ hoặc Google Cloud Run). Webhook của GitHub vẫn trở về FastAPI trên máy chủ đó. Postgres đang có vẫn là nơi lưu lịch sử và bảng dấu vân tay; Firestore chỉ nhận bản "tóm tắt để hiển thị" — tôi không định viết lại `memory.py`.

Kiến trúc dự kiến:



Các bước thử nghiệm (ước 1,5–2 tuần, xen kẽ với việc chính):

Bước	Việc	Tiêu chí xong
1	Tạo project Firebase, bật Hosting + Auth (GitHub) + Firestore; tạo OAuth App trên GitHub	Đăng nhập được bằng tài khoản GitHub, thấy tên người dùng
2	Worker ghi tiến độ: thêm <code>app/publish_firestore.py</code> (Admin SDK, service account) gọi sau mỗi bước review; tắt được bằng biến môi trường để eval không bị ảnh hưởng	Chạy CLI trên một case, Firestore hiện đúng chuỗi trạng thái và danh sách nhận xét
3	Trang web tối thiểu: danh sách PR, trang chi tiết, badge trạng thái realtime	Mở hai tab, chạy review ở tab này, tab kia tự đổi
4	Endpoint <code>/demo/review</code> có giới hạn tần suất và chỉ cho repo mẫu, để khách không tiêu hết hạn mức LLM	Gọi quá 3 lần/giờ bị từ chối; repo ngoài danh sách bị từ chối
5	Triển khai worker lên VPS/Cloud Run, nối webhook GitHub thật	Mở PR trên repo mẫu → comment xuất hiện trên GitHub và trên web

Rủi ro đã thấy trước: quyền Firestore phải khóa chặt (chỉ worker được ghi, khách chỉ đọc) — tôi sẽ viết security rules trước khi mở public; hạn mức LLM miễn phí là nút cổ chai nên demo cần cache kết quả của các PR mẫu. Nếu Firebase gây nhiều rắc rối hơn giá trị, phương án dự phòng là FastAPI phục vụ luôn trang tĩnh và đẩy trạng thái bằng SSE — mất Auth và HTTPS "sẵn", nhưng ít thứ phải học.

7. Tuần tới

- Bắt đầu bước 1–2 của kế hoạch Firebase (phần worker ghi tiến độ làm trước, vì nó độc lập với giao diện).
- Thử một model mạnh hơn (Claude hoặc Gemini Pro) trên cả ba bộ, trước hết là 15 PR thật, để biết hệ thống "im lặng" vì model hay vì lỗi nhắc. Đây là câu hỏi quan trọng nhất còn mở.
- Dùng model khác họ làm giám khảo phân loại báo động giả, hoặc tự rà tay một mẫu 20 nhận xét.
- Nếu có credential GitHub App: thử luồng thật trên repo demo, đo dấu vân tay có chặn đăng trùng không.

Phụ lục. Commit trong kỳ

Ngày	Commit	Nội dung
05/10	<code>5b6449a</code>	Sửa bước gộp static/LLM theo kết quả đo, lọc nhiễu bandit B101
05/10	<code>057fead</code>	Chạy lại kết hợp thật, nối bộ lọc B101 vào <code>collect()</code> (bug D3), backend Anthropic, tài liệu đề xuất
05/10	<code>d2c5a0b</code>	Triển khai 7 đề xuất từ khảo sát Open Code Review
06/10	<code>2133adf</code>	Chặn fact-check lý luận vòng, lọc static theo phạm vi thay đổi, diff AACR đúng chiều
06/10	<code>87c12e2</code>	Đề xuất: mục 9 kết quả sau triển khai
07/10	<code>8e87c2e</code>	Gộp cảnh báo tĩnh cùng dòng; gọi lại JSON hòng có kèm lỗi
07/10	<code>869687c</code>	Đề xuất: số đo cuối
07/10	<code>a881536</code>	Báo cáo tiến độ (nhóm E, bảng 7.6), <code>tools/md2pdf.py</code>

Tài liệu liên quan trong repo: `BAO_CAO_TIEN_DO.md` (21 trang PDF), `docs/DE_XUAT_CAI_TIEN_TU_OPEN_CODE_REVIEW.md` (9 trang PDF), `eval/README.md`.